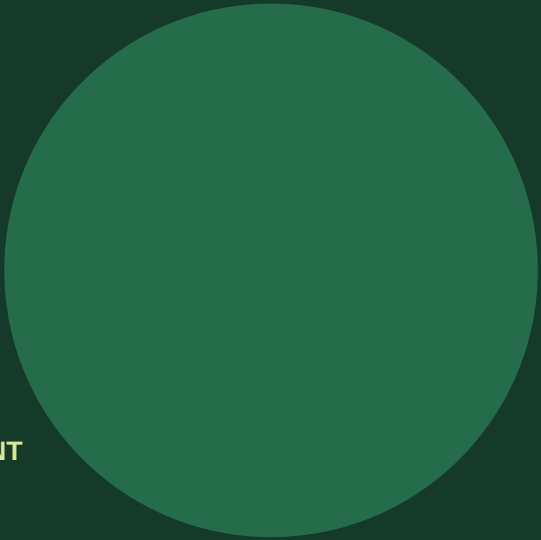




PRODUCT AND ENGINEERING BLUEPRINT

Kishan Bhaiya

Farm-to-market commerce, collective procurement, and capacity-aware smart logistics

Architecture report

Complete application purpose, roles, workflows, services, data model, API surface, security, deployment path, and production roadmap.

Prepared from the current codebase | 2026-08-23



Executive summary

Kishan Bhaiya is a role-governed agriculture ecosystem that links local producers, Farmer Producer Organisations (FPOs), retail buyers, business buyers, drivers, fleet partners and operations administrators. It is designed to make agricultural commerce more transparent from product discovery through to fulfillment, delivery, payments, and post-order resolution.

Core product idea

A buyer can discover trusted produce or raise a large requirement; verified producers can list supply, lots and future harvest; the platform can select and split supply across eligible sellers; and logistics operations can create capacity-safe trips, optimize stops, and consolidate compatible loads already in transit.

What the current implementation is

Area	Implementation
Application style	MERN modular monolith: React/Vite client, Express API, MongoDB system of record.
State and realtime	TanStack Query for server state, Zustand for safe device state, Socket.IO for authenticated cache invalidation and realtime enhancement.
Geospatial and logistics	Coordinates, Leaflet maps, Haversine distance, constrained nearest-neighbour route estimate, and capacity/cold-chain checks.
Trust boundaries	JWT access/refresh sessions, role and ACTIVE-status gates, validation, server-side totals, audit records, and private verification-document metadata.
Resilience	PWA app shell, IndexedDB listing drafts, explicit provider fallbacks, and development-only memory fallback when configured.

Business outcomes

- Reduce producer dependence on opaque intermediaries by exposing supply, inventory, buyer demand and order status.
- Let buyers obtain larger quantities without manually contacting many growers, while showing why a seller or split was chosen.
- Avoid avoidable spoilage through FEFO lot allocation, freshness alerts, pre-harvest reservations and surplus rescue offers.
- Improve vehicle utilization by planning capacity-safe multi-pickup trips and accepting compatible loads while a vehicle is already in transit.
- Give each actor only the work and data appropriate to their role, including scoped dashboards and private verification processes.

1. Real-world problems solved

Problem in agriculture	Kishan Bhaiya response	Measurable result
Fragmented smallholder supply	Producer listings, FPO aggregation, lot records and multi-seller fulfillment plans combine supply while retaining seller identity.	Higher fulfillment coverage and lower buyer coordination effort.
Price opacity and weak buyer trust	Seller details, ratings/reliability context, quality passports, price intelligence and explainable matching.	Better informed selection and traceability.
Manual B2B sourcing	Business buyers publish requirements; platform produces ranked candidates, quotations, counters and capacity-aware split plans.	Faster procurement cycle with documented negotiation.
Post-harvest waste	Lot expiry/freshness states, scheduled rescue candidates, promotional rescue offers and FEFO reservations.	More produce sold before quality declines.
Empty or under-used vehicles	Automatic trip construction, vehicle/driver compatibility checks and in-transit load opportunities.	Higher utilization and fewer duplicate trips.
Unclear delivery accountability	Ordered stops, proof capture, exception reports, dispute records, audit trails and controlled status transitions.	Stronger delivery evidence and faster recovery.
Digital access barriers	English, Hindi and Odia interface, mobile navigation, PWA support, low-bandwidth animation setting and current-location capture.	Broader rural and mobile usability.

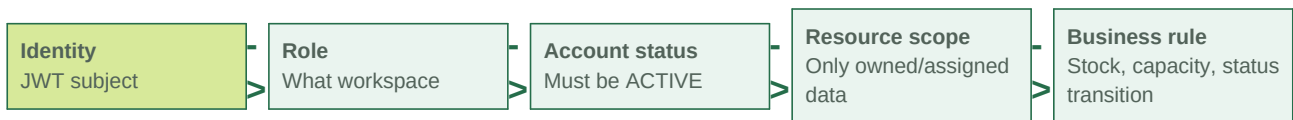
Design principle

The platform does not treat all users as generic shoppers. It separates commerce, producer operations, FPO aggregation, fleet execution and administration. This avoids accidental data exposure and prevents roles such as farmer, driver or admin from buying retail products merely because they can reach a public route.

2. Users, permissions and trust model

Role	Primary responsibilities	Marketplace access
Consumer	Browse, save produce, cart, checkout, track own orders, maintain profile and preferences.	Can browse and purchase retail.
Business buyer	All buyer actions plus bulk requirements, quotations, negotiation, recurring demand and multi-seller acceptance.	Can browse, purchase and procure.
Farmer	Create/manage produce and lots, publish expected harvest, offer surplus rescue, request FPO membership, see producer analytics.	Producer workspace; no buyer checkout.
FPO manager	Manage FPO catalog, members, aggregations and settlements.	Producer/FPO workspace; no buyer checkout.
Driver	See assigned trips only; start trip, follow next stop, submit proof, report issues and evaluate compatible in-transit loads.	Logistics execution only.
Fleet/logistics partner	Manage vehicles and dispatch, run planning, review shipment capacity and route estimates.	Logistics operations only.
Administrator	Review verification, resolve disputes, inspect audits and operational metrics.	Administration only.

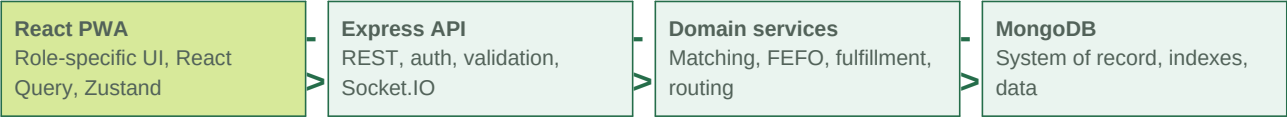
Authorization layers



- Registration assigns a role. Consumer accounts can be immediately active in the development flow; operational roles proceed through verification status.
- Express validates both role and account status before protected operations. A pending grower, business account or driver cannot use operational APIs until active.
- Resource checks add a second boundary: drivers are limited to shipments assigned to their user ID; producers are scoped to their seller-linked data; buyers see their own orders.
- The React navigation reflects the role, but the server is authoritative. Hiding a menu item is never relied upon as security.

3. Logical architecture

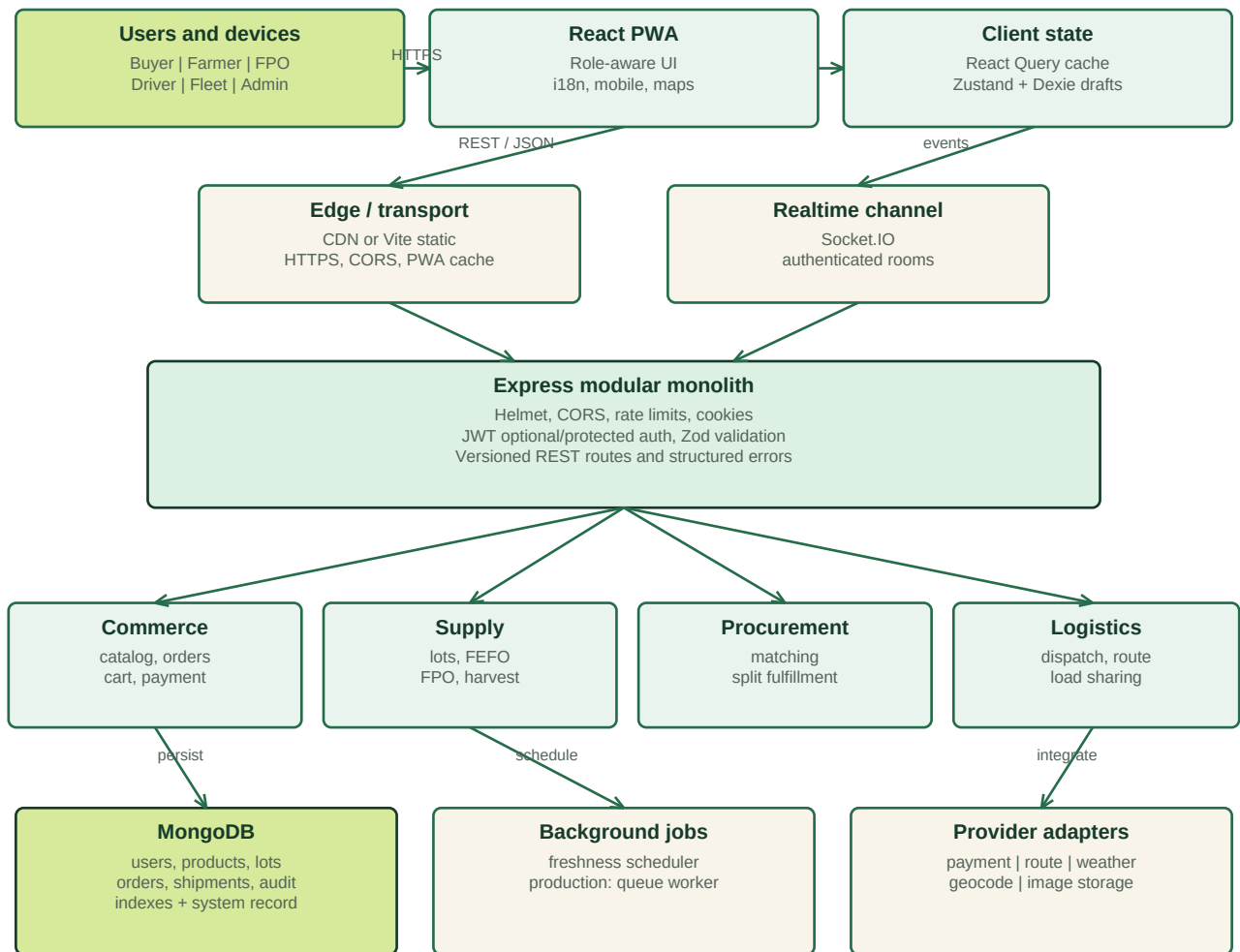
The codebase is a modular monolith. The modules are separated by responsibilities and provider interfaces, but the API deploys as one Express service. This is practical for a hackathon or early product: it reduces distributed-systems overhead while preserving clear seams for later extraction.



Layer	Responsibilities	Principal technologies
Presentation	Responsive role-aware screens, language resources, maps, charts, accessibility and purposeful motion.	React 19, React Router 7, Tailwind CSS, Framer Motion, Lucide.
Client state	Cache server records, invalidate after mutations/events, persist cart/session presentation/preferences safely.	TanStack Query 5, Zustand 5, Axios, Dexie IndexedDB.
Transport	Versioned JSON REST APIs, cookie/token support, realtime event channel.	Express 5, Socket.IO 4, CORS, HTTP.
Domain	Commerce, bulk procurement, lot allocation, FPO, dispatch, verification, dispute and analytics rules.	JavaScript ES modules, Zod validation, service layer.
Persistence	Operational data, users, products, lots, orders, shipments, verification, audit history.	MongoDB 7, Mongoose 8.
External adapters	Payment, route, weather, image upload and geocoding boundaries with labelled fallback behavior.	Razorpay-ready, OpenRouteService-ready, Cloudinary-ready, Nominatim.

Why this is extensible

The matching service exposes weights and explanations, while the route service returns a provider-compatible result. A Python/OR-Tools optimizer, queue worker, payment gateway or commercial routing engine can be introduced behind these contracts without rewriting the React application.



Security boundary: role + ACTIVE account status + resource ownership/assignment + server-side business checks.

Scale path: CDN/WAF -> stateless API replicas -> managed MongoDB replica set -> Redis/BullMQ workers -> provider services.

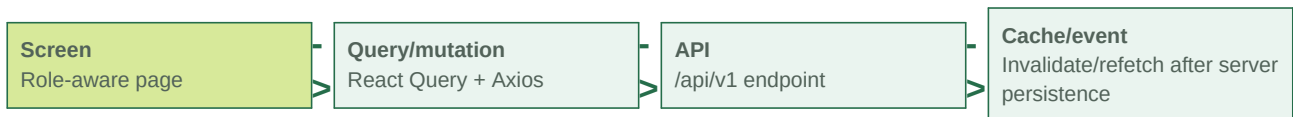
Reading the diagram

React remains responsible for user experience and cache state. Express owns authorization and business rules. Domain services contain matching, FEFO allocation, fulfillment and routing logic. MongoDB is the system of record; jobs and external providers are invoked through clear boundaries.

4. Frontend architecture

Client module	Role in system
src/api/client.js	Central Axios client and request/auth handling for the versioned API.
src/layouts	Public shell, dashboard shell and protected-route boundary.
src/pages	Authentication, marketplace, cart, checkout, producer, FPO, bulk, logistics, verification, profile and static screens.
src/components	Navbar, mobile bottom navigation, product cards, maps, realtime bridge, avatar, UI primitives and reusable motion.
src/store/useAppStore.js	Safe presentation state: session representation, cart drafts, language/preferences and low-bandwidth setting. It is not the source of truth for orders or inventory.
src/i18n	English, Hindi and Odia translation resources. Signed-in users change language from Profile > App preferences.
src/data/offlineDb.js	Dexie/IndexedDB persistence for offline listing drafts; it does not pretend that an order or reservation succeeded while offline.

Client request lifecycle



- The public and dashboard layouts compute navigation from the logged-in role. Mobile navigation offers the same core account actions, including logout, without relying on desktop menus.
- RealtimeBridge authenticates the Socket.IO connection when an access token exists. Events are treated as a signal to invalidate cached records, not as a replacement for persisted API truth.
- Framer Motion provides different, restrained animation intent: commerce cards reveal gently; operations screens sequence efficiently; active logistics trips receive a calm live pulse. Low-bandwidth and OS reduced-motion preferences suppress non-essential movement.
- Leaflet/React Leaflet renders locations and route/stops. Browser geolocation can obtain current location; geocoding supports India-wide place lookup through a provider boundary rather than a hard-coded Odisha-only list.

5. Backend architecture and API

Backend module	Responsibility
src/app.js	Express middleware pipeline: Helmet, CORS, JSON limits, cookies, static public uploads, auth rate limiting, optional auth, routes and structured errors.
src/server.js	Database initialization, optional demo seeding in development, HTTP server, Socket.IO authentication/rooms and development scheduler.
src/routes/api.js	Versioned REST resource surface and orchestration of permissions, persistence, service calls, audit and notifications.
src/middleware	JWT protection, optional authentication, validation, error serialization and multipart upload limits.
src/services	Data-store boundary, matching, fulfillment planner, load sharing, route optimization and geocoding.
src/providers	Configurable integration boundary for payment, routing, weather, pricing and uploads.
src/jobs/scheduler.js	Development freshness scheduling; production should replace critical scheduling with durable queue workers.

API conventions

All endpoints are under **/api/v1**. Successful responses use **{ success, data, meta? }**; error responses use **{ success: false, error }**. API inputs with structured requirements are validated with Zod. Mutating operations calculate totals, stock and reservations on the server.

Domain	Representative operations
Identity	register, login, refresh, logout, profile edit, profile photo and verification submission.
Commerce	list/detail/related products, seller public profile, cart-to-order checkout, buyer order tracking.
Bulk procurement	create requirement, rank matches, request quotes, counter/accept/reject, accept fulfillment plan.
Supply	create product, lots, price intelligence, expected harvest, harvest reservation/convert, rescue offers.
FPO	membership requests, members, aggregation and settlements.
Logistics	shipments, dispatch, start, optimize, complete stops, issue/proof, load opportunities and acceptance.
Trust	disputes, verification review and admin audit.

6. Data architecture

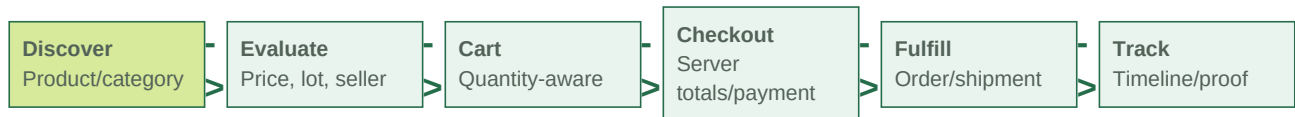
MongoDB is the system of record. Mongoose schemas define key fields and indexes; some related domain records use flexible schemas in the current modular-monolith implementation. Application IDs are string identifiers, making public IDs and seed records straightforward to manage.

Entity group	Key records	Important ownership/reference
Identity and trust	User, VerificationProfile, VerificationDocument, VerificationReview, Notification, AuditLog	User role/status; document owner; reviewer and applicant references.
Catalog and supply	Seller, Product, ProduceLot, QualityPassport, PriceSnapshot, RescueOffer	Product/lot sellerId; lot productId; passport lotId.
Commerce	Order, OrderSubFulfillment, Payment, Dispute	Order buyerId/sellerId; sub-fulfillments expose per-seller execution.
Bulk procurement	BulkRequirement, Quotation, Negotiation, ExpectedHarvest, HarvestReservation	Requirement buyer/product; quote seller; reservation harvest and buyer.
FPO	FPOProfile, FPOMember, FPOMembershipRequest, Aggregation, Settlement	FPO/member relationship and aggregation records.
Logistics	Shipment, ShipmentStop, Vehicle, DriverProfile, CollectionHub	Shipment orderIds, vehicle/driver assignment and ordered stops.

Data integrity strategy

- Indexes support common lookups such as role/status, product/seller, lots/expiry, order buyer/seller, shipment status and notification read state.
- Lot allocation is FEFO: earlier-expiring eligible inventory is selected before later-expiring stock, while unavailable/expired inventory is excluded.
- For production MongoDB replica sets, quotation acceptance and reservation/order creation should run inside a transaction to make multi-document state changes atomic.
- AuditLog and VerificationReview preserve who initiated sensitive decisions and the previous/next status, improving operational traceability.

7. Retail commerce workflow



Step	What happens	Control
1. Browse	Only consumer and business-buyer roles are allowed into buyer commerce. Product discovery includes related and recently viewed produce.	Role-aware routes and server-side purchasing permission.
2. Trust check	Buyer can see public-safe seller information, reliability context, inventory summary, pricing and related products.	Private verification evidence is never exposed via public seller views.
3. Cart	Cart quantities are tracked locally for convenience, but the server remains authoritative for price and availability.	Quantity and threshold validation.
4. Checkout	API calculates totals, checks inventory and creates the order. Payment uses a Razorpay-ready boundary or explicitly labelled mock behavior in development.	No client-calculated final price is trusted.
5. Fulfillment	Inventory movement/order updates create records that logistics can use for dispatch and tracking.	State transition and stock checks.
6. After sales	Buyer views own orders and can create a dispute with context/evidence where applicable.	Ownership scope and auditability.

Retail limitation by design

Farmers, FPO managers, drivers, fleet partners and administrators do not receive buyer checkout actions. They see their operational workspaces instead. This matches the requested role separation and prevents unauthorized ordering.

8. Bulk procurement and multi-seller fulfillment

Bulk buying is a structured B2B workflow rather than a larger shopping cart. A business buyer declares the product, quantity, location, date, target price, quality and partial-fill preference. The platform produces candidates and, when needed, a transparent supplier split.

Candidate scoring

Factor	Weight	Interpretation
Landed price	30%	Seller price plus distance-informed transport estimate compared to buyer target.
Distance	20%	Haversine distance between supply and requirement coordinates.
Coverage	20%	Share of requested quantity available from the seller.
Reliability	15%	Seller fulfillment reliability indicator.
Freshness	10%	Lot freshness state prioritizes fresh supply.
Quality	5%	Product grade compared with requested quality.

Fulfillment workflow



- If full fill is required, the plan chooses an eligible single supplier that can cover the requirement. Otherwise it can allocate sequentially across ranked sellers until filled or supply runs out.
- Each allocation records quantity, allocation percentage, subtotal, estimated transport and split reason. The buyer can see why the split occurred rather than receiving an unexplained recommendation.
- Acceptance reserves exact eligible lots using FEFO and creates order sub-fulfillments so each seller can execute only their portion.
- Shortage reporting preserves an audit event and can calculate an alternate-supplier recovery plan for explicit buyer approval.

9. Supply, freshness and FPO workflows

Workflow	Business flow	Value
Producer listing	ACTIVE farmer/FPO manager creates product and supply lots with availability, grade/location and freshness information.	Creates a trusted digital catalog and orderable inventory.
Lot management	Lots carry expiry/freshness. Eligible lots are selected FEFO for supply reservations.	Reduces waste and prevents reserving unsuitable stock.
Pre-harvest	Producer publishes expected harvest; buyer makes conditional reservation; producer converts harvest into actual lot when available.	Allows forward planning without falsely treating future yield as current stock.
Surplus rescue	Freshness scheduler identifies at-risk stock; producer confirms a promotional rescue price; offer is persisted to marketplace context.	Moves sell-soon produce before loss.
FPO membership	Farmer submits join request; FPO manager approves/rejects; approved member enters aggregation and settlement workflow.	Digitizes producer collective participation.
Settlements	FPO views member/aggregation settlement records by organization scope.	Improves transparency for pooled supply.

Scheduled freshness handling

A development scheduler runs freshness logic in-process. It is appropriate for demo and single-process development. Production should invoke the same idempotent domain operation through a durable, observable queue such as Redis/BullMQ, with retry policy and dead-letter handling.

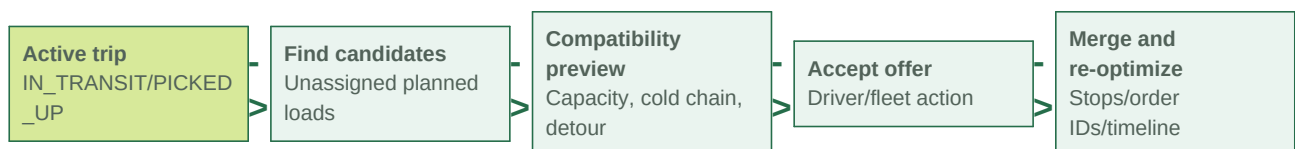
10. Smart logistics, routing and load consolidation

Logistics begins when an accepted multi-seller plan produces supplier allocations. The fulfillment planner turns those allocations into capacity-safe shipment drafts. It selects compatible available/idle vehicles and active drivers, constructs pickup/hub/delivery stops, and calls the route optimizer.

Automatic dispatch design

Rule	Current implementation
Vehicle capacity	A draft uses available vehicle capacity. Segments are split across trips when supply exceeds available trip capacity.
Cold chain	If any segment needs cold chain, compatible cold-chain vehicles are selected; incompatible configuration produces a warning.
Driver eligibility	Only ACTIVE driver users with no current shipment are candidates. Assigned driver is attached to the trip.
Stop design	Pickup stops are grouped by seller. A collection hub is added for multi-pickup trips when hub data is available, followed by delivery.
Route order	Completed stops remain fixed. Pending stops are ordered with pickups first, then hubs, other stops and deliveries, using nearest-neighbour selection within each group.
Estimate	Distance is Haversine-based; duration assumes average 32 km/h plus 18 minutes per intermediate handling stop. Fuel estimate varies by cold-chain use.

In-transit spare-capacity workflow



- An active trip must be IN_TRANSIT or PICKED_UP. A candidate must be a planned/ready unassigned load; a load already offered elsewhere cannot be taken.
- The system rejects invalid quantities, capacity overflow, cold-chain incompatibility and excessive detours. The safe detour threshold is max(20 km, 50% of existing route distance).
- On acceptance, candidate order IDs, segments and stops are merged into the active shipment; capacity utilization, route, timeline and acceptance audit fields are updated.
- This enables the requested practical behavior: a driver/fleet can accept a compatible additional load only when spare capacity and route constraints permit it.

11. Verification, security and privacy

Security area	Implemented approach	Production hardening
Authentication	JWT access token contains subject, role, name and account status. Refresh token is separately hashed and expires by configured TTL.	Short access TTL, key rotation, secure cookie settings, session-reuse detection.
Authorization	Role gate plus ACTIVE account enforcement plus ownership/assignment checks in API routes.	Centralized policy engine and permission tests for every endpoint.
HTTP safety	Helmet, disabled x-powered-by, configured CORS credentials/origins, auth rate limiter, body size limits and structured error handler.	WAF, stricter rate limits, CSP review and security monitoring.
Input/file safety	Zod structured validation; profile image type/size limits; verification evidence remains private metadata and does not expose secure file keys publicly.	Malware scanning, signed URLs, encrypted object storage and retention rules.
Data integrity	Server computes order totals and inventory availability; status transitions and high-impact actions create audit context.	MongoDB transactions, outbox pattern and immutable audit storage.
Privacy	Only public-safe seller details are shown. Driver phone is masked until assignment; documents do not become public profile assets.	Consent records, data export/deletion workflow and formal privacy assessment.

Important secret-management note

Environment values must remain outside source control. Rotate any credential that was ever pasted into a document, chat, repository or example configuration, then replace it using the deployment secret manager. This report intentionally does not reproduce keys, passwords, tokens or connection strings.

12. Deployment, operations and scalability

The current application can run locally with a Vite development server, Express API and MongoDB. Docker Compose supplies a MongoDB 7 development service. It also supports a clearly labelled development-only in-memory fallback when MongoDB is unavailable; this fallback resets on restart and is never a production database strategy.

Environment	Recommended topology
Local development	React/Vite on port 5173, Express/Socket.IO on port 5000, Docker MongoDB or dedicated demo MongoDB database, non-destructive fictional demo seed.
Staging	Static client build behind CDN, containerized API, managed MongoDB replica set, object storage for uploads, observability and provider sandbox keys.
Production baseline	CDN/WAF -> load balancer -> stateless API replicas -> managed MongoDB replica set. Redis/BullMQ workers for jobs, queue fan-out and retries. Cloud object storage/signed URLs for files.
Scale-out path	Extract route optimization, notification delivery and heavy report/analytics workloads only when their traffic/compute warrants it. Keep APIs contract-driven.

Operational observations

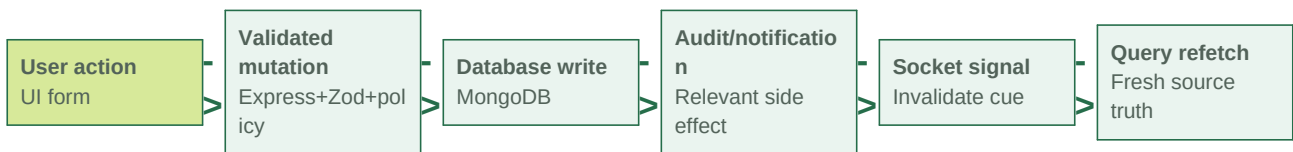
- A port-in-use error (EADDRINUSE on 5000) means another server already owns that port. Stop the duplicate process or choose a different configured API port; it is not a database error.
- Login HTTP 500 errors should be diagnosed from API logs and database connectivity. The application's development memory fallback may let the API start, but persistent account data requires a successful MongoDB connection.
- Monitor: API error rate/latency, MongoDB connection pool, order/reservation failures, queue depth, route plan duration, verification turnaround, upload failures and Socket.IO connection count.
- Back up MongoDB, test restoration, isolate tenant/demo data, and configure provider timeouts/circuit breakers before production use.

13. End-to-end workflow map

The following shows how the principal domain workflows connect. Every mutation goes to Express; client cache and realtime events improve the experience but do not bypass server persistence.

Journey	Start	Core decision	Persistent outcome
Consumer purchase	Buyer discovers product	Stock, server total and payment state validated	Order, payment reference, fulfillment/shipment tracking.
Business procurement	Buyer posts requirement	Weighted candidate score and fill policy determine plan	Quotes/negotiation, FEFO reservations, order sub-fulfillments and shipment drafts.
Producer supply	Producer publishes product/lot or expected harvest	ACTIVE status and supply/freshness rules	Catalog inventory, conditional reservation or rescue offer.
FPO joining	Farmer requests membership	FPO manager approves/rejects	Membership access to aggregation/settlement workflows.
Auto logistics	Plan accepted	Vehicle capacity, driver activity, cold chain and stop order	Ready/planned shipments with optimized route estimate.
In-transit consolidation	Active driver/fleet finds extra load	Capacity, detour, status and cold-chain compatibility	Merged active shipment, re-optimized route, timeline/audit acceptance.
Exception/dispute	Issue or buyer dispute raised	Authorized review and evidence/status rules	Audit trail, resolution state and notification.

Event and cache sequence



Failure behavior

The UI should never claim that a payment, order or reservation is complete merely because a client request was started or an offline draft was saved. Failures return an API error, preserve server truth, and can trigger a visible recovery workflow such as alternate supply review.

14. Technology inventory

Category	Technology	Purpose
Language/runtime	JavaScript ES modules, Node.js 20+	Shared full-stack language and server runtime.
Web client	React 19, Vite 7, React Router 7	Fast SPA development, routing and optimized production bundle.
UI	Tailwind CSS, clsx, tailwind-merge, Lucide React, Framer Motion	Responsive styling, icons and controlled contextual motion.
Data/UI forms	TanStack React Query, Axios, React Hook Form, Zod	Remote cache, HTTP, forms and validation.
Localization	i18next, react-i18next	English, Hindi and Odia user interface resources.
Maps/analytics	Leaflet, React Leaflet, Turf helpers, Recharts	Location, route/map display and dashboard visualization.
Offline/PWA	vite-plugin-pwa, Dexie	App shell caching and IndexedDB drafts.
API	Express 5, Socket.IO, CORS, Helmet, Morgan, express-rate-limit	REST/realtime, request safety, logging and rate limiting.
Identity/files	jsonwebtoken, bcryptjs, cookie-parser, Multer	Password hashing, token sessions, cookies and controlled multipart uploads.
Database/jobs	MongoDB 7, Mongoose 8, node-cron	Persistence, schemas/indexes and development freshness jobs.
Testing	Vitest, Supertest, Testing Library, jsdom, ESLint	API/business tests, component tests and static quality checks.
External-ready adapters	Razorpay, Cloudinary, OpenRouteService, weather/geocoding providers	Integration seams with explicit mock fallback state.

15. Production roadmap

Priority	Next improvement	Why it matters
P0	Rotate exposed credentials; use managed secrets; verify no secrets are committed; enforce production environment validation.	Protects database, tokens and third-party accounts.
P0	Use a managed MongoDB replica set and transactions for multi-document reservations/order acceptance.	Prevents partial fulfillment state under concurrency.
P0	Move jobs, notifications and route fan-out to Redis/BullMQ workers with retries and idempotency keys.	Makes background work reliable under replica scaling.
P1	Integrate real payment, maps/directions, route distance matrix and weather providers with quotas, failure policies and user-facing truth labels.	Improves operational accuracy without pretending mock data is live.
P1	Store uploads in encrypted cloud object storage with malware scanning and signed URLs.	Improves privacy and verification evidence handling.
P1	Add observability: structured logs, trace IDs, metrics, error reporting, business KPI dashboards and alert rules.	Enables fast 500-error diagnosis and safe operations.
P2	Evolve optimizer to OR-Tools/time windows/vehicle classes and run simulations using actual transport history.	Improves cost, fill rate and delivery ETA at scale.
P2	Add SMS/WhatsApp/voice notifications and assisted workflows for low-connectivity users.	Improves rural adoption and completion rates.
P2	Formalize data-retention, consent, support and dispute SLA policies.	Builds long-term trust and compliance readiness.

Conclusion

Kishan Bhaiya is more than a catalog: it is a governed agriculture operating system that connects supply discovery, trusted trade, collective procurement and practical logistics. Its current modular-monolith architecture is well suited to product validation while keeping interfaces ready for production-grade providers and selective service extraction.

Recommended presentation summary

Kishan Bhaiya helps buyers source verified produce, helps farmers/FPOs sell and coordinate supply, and helps logistics teams deliver more efficiently through capacity-aware planning and in-transit load consolidation. The platform maintains transparency at every hand-off through role-based access, inventory/lot controls, explanations and audit records.